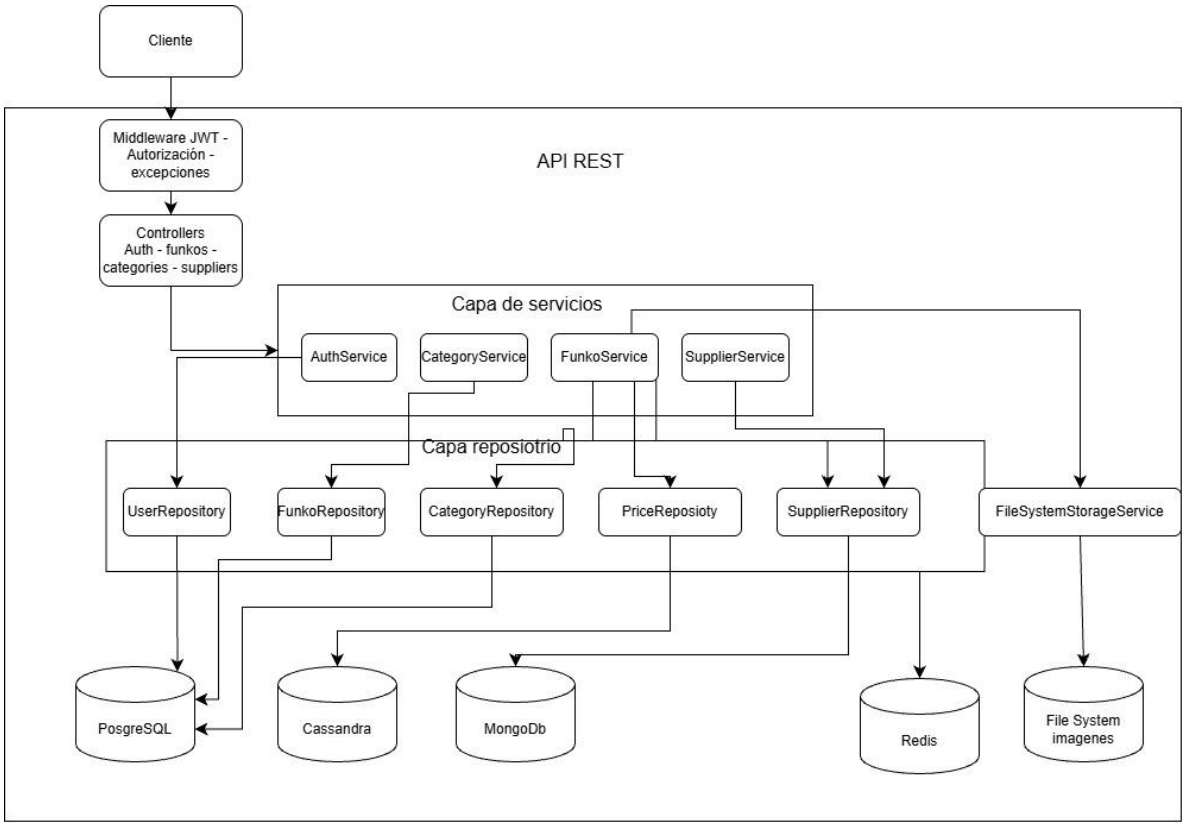
FUNKO API

Desarrollo Web En Entorno Servidor



IES Luis Vives
Kevin Zelaya – Sergio Burda

Contenido

1 Objetivo de la práctica 2

2 Descripción general del sistema. 2

3 Requisitos funcionales. 3

4 Requisitos No funcionales. 4

5 Requisitos de información. 5

6 Tecnologías utilizadas 6

7 Arquitectura y organización del código. 7

8 Estructura del proyecto. 8

9 Persistencia de datos. 9

10 Seguridad 11

11 Modelo de datos 11

11.1 Validación de datos 13

11.2 Relaciones entre clases 14

11.3 Paginación 14

12 Comportamiento del sistema 15

13 API REST y endpoints 16

13.1 AuthController 16

13.2 FunkoController 16

13.3 SupplierController 17

13.4 CategoryController 18

14 Control de concurrencia. 18

15 Servicio de almacenamiento de imágenes 19

16 Diagramas de secuencia – CRUD 19

17 Testing y pruebas 22

18 Documentación del código 23

19 Conclusión 28

1 Objetivo de la práctica

El objetivo de esta práctica es poner en práctica y consolidar gran parte de los conocimientos adquiridos a lo largo del segundo tema de la asignatura de Entorno Servidor. Para ello, se desarrollará una **API REST** aplicando las tecnologías, patrones de diseño y convenciones estudiadas durante el tema, siguiendo buenas prácticas de desarrollo y una arquitectura organizada por capas.

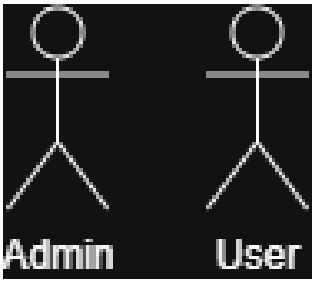
Durante el desarrollo se trabajarán conceptos fundamentales como la **inyección de dependencias (DI)**, la separación de responsabilidades, la programación asíncrona, el uso de interfaces, los patrones Repository y Service, la gestión centralizada de excepciones y la validación de datos. También se buscará construir una aplicación modular, mantenible y preparada para facilitar futuras ampliaciones.

Uno de los principales objetivos de la práctica será integrar diferentes sistemas de almacenamiento y fuentes de datos, concretamente **PostgreSQL, Redis, Cassandra y MongoDB**, aprendiendo a trabajar con sus particularidades y aprovechando las características que ofrece cada tecnología. De esta manera, la API deberá ser capaz de consultar, almacenar y gestionar información procedente de diferentes sistemas, combinando los resultados cuando sea necesario.

Además, se trabajará con la comunicación entre las distintas capas de la aplicación, la configuración de los servicios y la gestión de las operaciones de acceso a datos, procurando que todas las operaciones se realicen de forma eficiente y asíncrona.

2 Descripción general del sistema.

La aplicación expone una API REST compuesta por cuatro controladores: **Funkos, Suppliers, Categories y Auth**. Todos ellos están protegidos mediante middleware de autenticación basado en JWT y un sistema de autorización por roles **(`admin` y `user`)**. Esto proporciona una capa de seguridad y permite establecer diferentes niveles de acceso a los distintos endpoints de la aplicación.

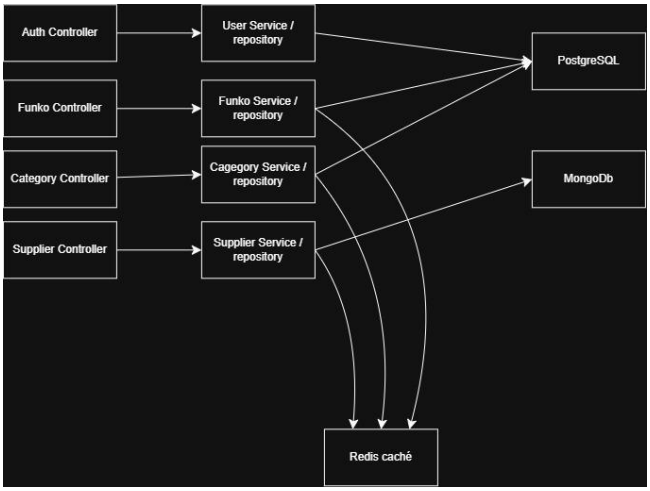


El sistema trabaja con cuatro tecnologías de bases de datos: una relacional, PostgreSQL, y tres no relacionales: Redis, Cassandra y MongoDB. Este enfoque permite gestionar la

información desde diferentes perspectivas y aprovechar las características de cada tecnología para optimizar las consultas y el acceso a datos.



En el caso de PostgreSQL y Redis, su utilización responde principalmente a la necesidad de mejorar el rendimiento y facilitar determinadas consultas mediante el almacenamiento en caché. Por otro lado, la separación de la información entre PostgreSQL y MongoDB responde, en mayor medida, a un objetivo práctico y académico, ya que no se trata de una separación estrictamente necesaria desde el punto de vista arquitectónico, sino de un requisito establecido en la prueba para añadir complejidad al desarrollo.



3 Requisitos funcionales.

Id	Requisito	Descripción
RF-01	Autenticación de usuarios	Permite registrar usuarios e iniciar sesión mediante credenciales, proporcionando un token JWT para acceder a los recursos protegidos.
RF-02	Autorización por roles	Controla el acceso a los recursos de la API mediante permisos asociados a los roles de usuario (Admin y User).
RF-03	Gestión de Funkos	Permite crear, consultar, actualizar y eliminar Funkos del sistema.
RF-04	Gestión de categorías	Permite administrar las categorías asociadas a los Funkos

RF-05	Gestión de proveedores	Permite registrar, consultar, modificar y eliminar proveedores, así como recuperar su información asociada a los Funkos.
RF-06	Gestión de imágenes	Permite almacenar, consultar y gestionar las imágenes asociadas a los Funkos.
RF-07	Búsqueda y filtrado	Permite localizar Funkos mediante criterios de búsqueda y filtrado.
RF-08	Paginación de resultados	Permite consultar los registros de forma paginada, estableciendo el número de página y la cantidad de elementos por página.
RF-09	Control de concurrencia	Permite detectar modificaciones simultáneas de una misma entidad para evitar la sobrescritura de información actualizada.
RF-10	Integración mediante API REST	Permite a sistemas externos interactuar con la aplicación mediante endpoints públicos y protegidos

4 Requisitos No funcionales.

Id	Requisito	Descripción
RF-02	Caché distribuida	Implementa Redis como mecanismo de almacenamiento temporal para mejorar el rendimiento y reducir las consultas a las bases de datos.
RF-03	Manejo global de excepciones	Dispone de un middleware global que captura excepciones no controladas y devuelve respuestas HTTP estandarizadas.
RF-04	Inyección de dependencias	Utiliza el contenedor de dependencias de ASP.NET Core para gestionar los servicios y repositorios de la aplicación.
RF-05	Arquitectura por capas	Separa las responsabilidades de la aplicación en diferentes capas para facilitar su mantenimiento, evolución y escalabilidad.
RF-06	Registro de eventos	Utiliza Serilog para registrar eventos y errores de la aplicación mediante salidas a consola y ficheros.
RF-07	Validación de datos	Aplica mecanismos de validación para garantizar que los datos recibidos cumplen las restricciones establecidas.

RF-08	Documentación de la API	Expone la documentación de los endpoints mediante OpenAPI y Swagger.
RF-09	Despliegue contenerizado	Permite desplegar la aplicación y sus servicios asociados mediante Docker y Docker Compose.
RF-10	Seguridad de las comunicaciones	Utiliza HTTPS para proteger las comunicaciones entre los clientes y la API.

5 Requisitos de información.

El manejo de los datos se realiza mediante modelos que representan las distintas entidades del negocio, como usuarios, Funkos, categorías y proveedores. Estos modelos permiten estructurar y gestionar la información dentro de la aplicación, mientras que los DTOs se encargan de definir los datos que se reciben y se envían a través de la API, controlando así el intercambio de información con los clientes.

Id	Requisito	Descripción
RF-01	Usuarios	Almacenar identificador, nombre de usuario, correo electrónico y contraseña cifrada.
RF-02	Funkos	Almacenar nombre, franquicia, número, precio, exclusividad e imagen
RF-03	Categorías	Almacenar identificador y nombre de cada categoría
RF-05	Arquitectura por capas	Separa las responsabilidades de la aplicación en diferentes capas para facilitar su mantenimiento, evolución y escalabilidad.
RF-06	Proveedores	Almacenar identificador y datos identificativos de los proveedores
RF-07	Historial de precios	Almacenar los precios registrados de los Funkos para consultar su evolución.

Por último, por convención, todas las entidades comparten una serie de campos de auditoría: fecha de creación, fecha de última actualización y fecha de eliminación. Además, cuentan con un campo booleano que indica si el registro ha sido eliminado. Esto permite llevar un seguimiento de los cambios realizados y aplicar un mecanismo de borrado lógico (Soft Delete), evitando eliminar físicamente los registros de la base de datos.

6 Tecnologías utilizadas

Tecnlología	Uso
.Net 10	Framework principal de la aplicación
ASP.NET Core	Framework para aplicaciones web
Entity framework Core	ORM para acceso y persistencia de datos
PostgreSQL	Base de datos relacional.
MongoDb	Base de datos documental base JSON
Redis	Caché distruibuilda.
Cassandra	NoSQL orientada a columnas anchas
JWT	Tecnología utilizada para la autenticación y transmisión de información de identidad.
Bcrypt	Algoritmo de hashing utilizado para proteger las contraseas.
FluenValidation	Biblioteca para implementar validaciones de datos mediante reglas.
NUnit	Framework para realizar pruebas unitarias.
Moq	Biblioteca para crear objetos simulados en pruebas unitarias.
CSharpFuncitonalExtensions	Implementacion del patrón Result para gestionar operaciones exitosas y errores.
GitHub	Control de versiones y alojamientos del código fuente.

7 Arquitectura y organización del código.

La aplicación sigue una arquitectura basada en la separación de responsabilidades. Los controladores (Controllers) se encargan de recibir y gestionar las peticiones HTTP, delegando la lógica de negocio en los servicios correspondientes (Services). Estos, a su vez, interactúan con los repositorios (Repositories) para consultar, almacenar y actualizar la información en las distintas fuentes de datos.

Tecnlología	Uso
Controller	Recibe peticiones HTTP, valida en primera instancia y devuelve respuestas HTTP.
Services	Servicios específicos para cada entidad, permite gestionar las reglas de negocio.
Repository	Gestionar la persistencia de usuarios mediante EF Core
Config	Modelos definidos para representar configuraciones específicas del appsettings.
Infraestructure	Inicialización e inyección de los diferentes componentes de la aplicación.
Models	Entidades que conforman el negocio.
Dto	Objetos para el manejo y transporte de datos.
Mappers	Permite la conversión entre modelos a DTOS
Middleware	Define middleware de control para el sistema.
Storage	Sistema para persistencia de imágenes.
Entities	Definición de contextos para las bases de datos e inicialización de las mismas.

8 Estructura del proyecto.

```
MinimalApi/
├── configuration/
├── controllers/
├── dto/
│   ├── auth/
│   ├── categories/
│   ├── funkos/
│   └── suppliers/
├── entities/
│   └── contextconf/
├── errors/
├── extensions/
├── infraestructure/
│   └── TimeLifes/
├── mappers/
├── middleware/
├── models/
├── repositories/
│   ├── Categories/
│   ├── funkos/
│   ├── suppliers/
│   └── users/
├── services/
│   ├── cache/
│   ├── categories/
│   ├── funkos/
│   ├── jwt/
│   └── suppliers/
├── storage/
├── validators/
│   ├── dtos/
│   └── fluent/
└── wwwroot/
    └── uploads/
        └── funkos/
```

En la siguiente estructura se muestran los principales directorios de la aplicación y su organización por responsabilidades, incluyendo los subdirectorios existentes. Se han omitido los archivos individuales para facilitar su visualización.

9 Persistencia de datos.

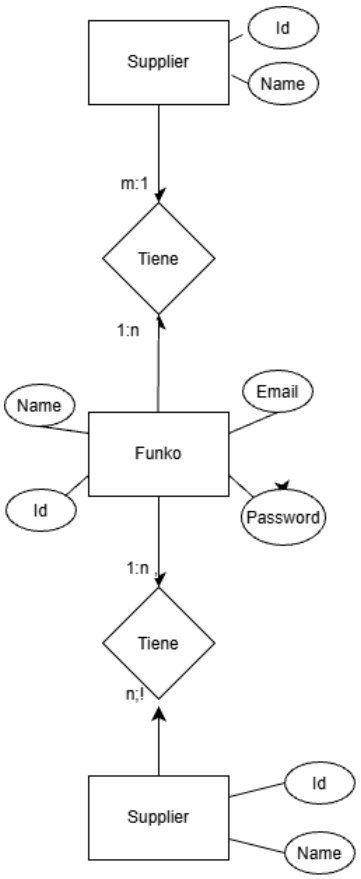
Para gestionar el almacenamiento de la información, la aplicación utiliza cuatro fuentes de datos diferentes, cada una con una finalidad específica y un nivel de responsabilidad determinado dentro del sistema. Esta arquitectura permite distribuir los datos en función de su naturaleza y de las necesidades de acceso, persistencia y rendimiento de cada componente.

En lugar de centralizar toda la información en una única base de datos, se aprovechan las características de distintas tecnologías, combinando bases de datos relacionales y NoSQL, así como sistemas de almacenamiento orientados a mejorar el rendimiento y la disponibilidad de la información.

A continuación, se describe el propósito de cada una de estas fuentes de datos y la función que desempeña dentro de la aplicación.

PostgreSQL: Es la principal base de datos relacional de la aplicación y se encarga de almacenar la información estructurada y persistente del sistema. En ella se gestionan los usuarios que pueden autenticarse en la aplicación, así como los Funkos y las categorías asociadas.

Al tratarse de una base de datos relacional, permite establecer relaciones entre las distintas entidades y mantener la integridad de los datos mediante restricciones y claves primarias y foráneas. De esta forma, garantizamos la consistencia de la información y facilitamos la realización de consultas y operaciones sobre los datos.



Además, utilizamos Entity Framework Core como ORM para interactuar con PostgreSQL, lo que nos permite trabajar con las entidades de C# sin tener que escribir manualmente todas las consultas SQL.

MongoDB: Es una base de datos NoSQL orientada a documentos que utilizamos principalmente para almacenar la información de los proveedores (Suppliers). Su elección responde tanto a sus características de flexibilidad como a la posibilidad de distribuir la información entre diferentes sistemas de almacenamiento según las necesidades de cada entidad.

En nuestro caso, los proveedores constituyen una entidad relativamente sencilla, compuesta principalmente por datos identificativos. Esta separación se debe inicialmente a un

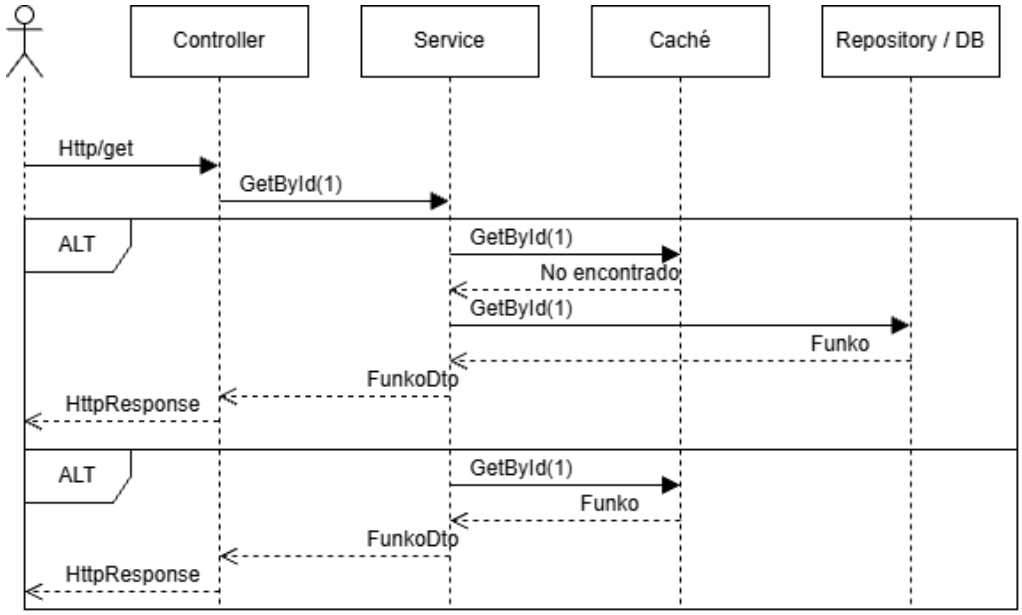
requisito práctico de la aplicación, pero también permite demostrar cómo es posible utilizar diferentes tecnologías de almacenamiento dentro de un mismo sistema, asignando a cada una responsabilidad concreta.

Para interactuar con MongoDB utilizamos Entity Framework Core mediante su proveedor para Mongodb, lo que nos permite trabajar con las entidades y realizar operaciones utilizando un enfoque similar al de las bases de datos relacionales. De esta forma, podemos mantener una estructura de acceso a datos familiar y aprovechar parte de las abstracciones que proporciona el ORM.

Pero, esta integración presenta ciertas limitaciones respecto a su utilización con bases de datos relacionales. Entre ellas encontramos diferencias en el seguimiento de entidades, la gestión de relaciones entre documentos y el soporte de transacciones. Estas particularidades hacen necesario tener en cuenta las características propias de Mongodb durante la implementación de los repositorios y las operaciones sobre los datos.

Redis: Es un sistema de almacenamiento en memoria que utilizamos como caché para guardar datos de consulta frecuente. Su objetivo es mejorar el rendimiento de la aplicación, reduciendo los accesos y la carga de las bases de datos principales.

Actúa como primera capa de consulta: antes de acceder a las bases de datos, el sistema comprueba si la información solicitada está disponible en Redis. Si no está almacenada, obtiene los datos de la fuente correspondiente.



10 Seguridad

El sistema está protegido mediante un mecanismo de autenticación basado en JWT (JSON Web Token), que permite controlar el acceso a las rutas de los controladores y proteger los distintos endpoints de la API.

Para acceder a los recursos protegidos, el cliente debe incluir el token JWT en cada petición HTTP, normalmente mediante la cabecera **Authorization**, utilizando el esquema Bearer. El servidor valida el token y comprueba su validez antes de permitir el acceso al recurso solicitado. De esta manera, se garantiza que únicamente los usuarios autenticados puedan acceder a los endpoints que requieren autorización.

Ejemplo petición por curl

```
curl --request POST \
  --url http://localhost:5205/api/funkos \
  --header 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxIiwiZW1haWwiOiJr
ZXZpbkBjb3JyZW8uY29tIiwianRpIjoiNmM2OTlmNDgtZTUyMC00NTM5LTgwZTctNjNlZDI1MWQ1NjE0IiwiaWF0IjoxNzkzNjk
1MDAwLCJ1c2VybmFtZSI6ImFkbWluIiwicm9sZSI6IkFETUlOIiwiZXhwIjoxNzkzNjk1OTAwLCJpc3MiOiJQcnVkdWN0b3NBcG
kiLCJhdWQiOiJQcm9kdWN0b3NBcGlDbGllbnRzIn0.q5Vb4OtRP0O9v43q9kOulIX27Ar1JqQWxf7bIsIgWS4' \
  --header 'Content-Type: application/json' \
  --data '{
  "name": "Gata mahoso",
  "franchise": "Marvel",
  "number": 123,
  "supplierId": "6ae9062dde1a14f83138128b",
  "categoryId": "7G7yFhPcC2l",
  "exclusive": false,
  "price": 14.99
}'
```

La verificación se realiza mediante el middleware de autenticación y autorización, situado en la canalización de procesamiento de peticiones HTTP antes de que estas lleguen a los controladores. Este mecanismo permite validar el token JWT y comprobar que el usuario dispone de los permisos necesarios para acceder al recurso solicitado.

Además, mediante la asignación de roles se puede restringir el acceso a determinados endpoints en función del nivel de permisos del usuario (ADMIN o USER). De esta forma, las operaciones más sensibles quedan reservadas a los usuarios con mayores privilegios, mientras que se limita el acceso al resto de usuarios según sus permisos.

11 Modelo de datos

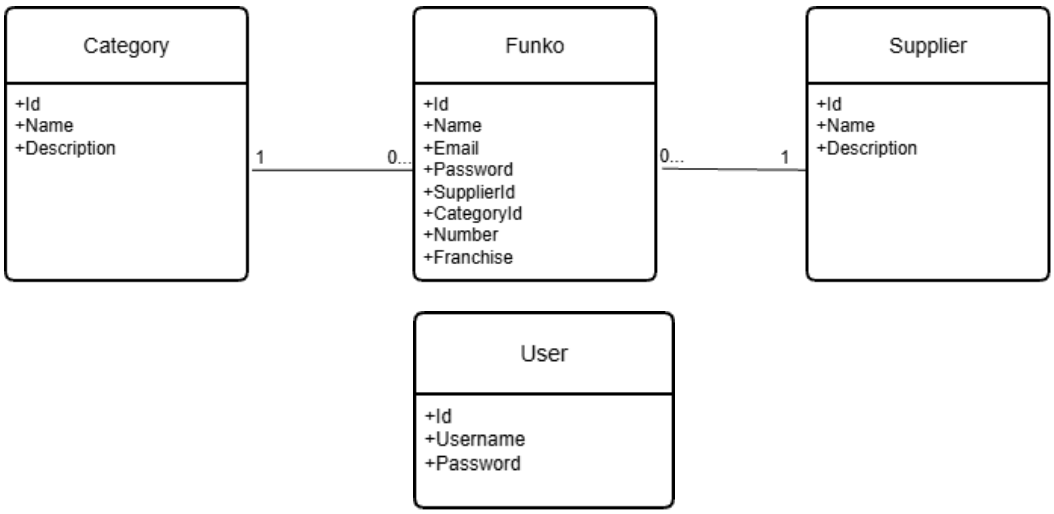
En cuanto a las entidades, el sistema trabaja con cuatro principales: Users, Funkos, Suppliers y Categories. Cada una cuenta con sus respectivos componentes funcionales, organizados en controladores (Controllers), servicios (Services) y repositorios (Repositories).

Esta separación permite distribuir las responsabilidades de cada entidad: los controladores gestionan las peticiones HTTP, los servicios contienen la lógica de negocio y los repositorios se encargan del acceso y la persistencia de los datos. De esta forma, se facilita la gestión de la información, la incorporación de nuevos registros y el mantenimiento de la aplicación.

En cuanto a los modelos, hemos decidido utilizar una única clase para representar tanto las entidades de la aplicación como los datos que se guardan en las bases de datos. Esto se debe a que los modelos son bastante sencillos y no necesitamos hacer conversiones ni mapeos entre diferentes estructuras.

Así evitamos tener clases duplicadas y podemos utilizar el mismo modelo en las distintas capas de la aplicación para consultar, modificar y guardar los datos.

Modelos:



Las tres entidades principales se relacionan mediante asociaciones, aunque la forma de establecer estas relaciones varía según la base de datos en la que se almacenan.

En el caso de Category y Funko, la relación se establece mediante una clave foránea (Foreign Key) y las restricciones correspondientes de PostgreSQL. Esto permite mantener la integridad referencial y aprovechar las funcionalidades de Entity Framework Core para incluir los datos relacionados mediante operaciones como Include(), obteniendo la información en una misma consulta.

```
public async Task<Funko?> GetFunkoByIdAsync(
long id,
    CancellationToken ct = default)
{
    Funko? funko = await _context.Funkos
        .AsNoTracking()
        .Include(f => f.Category)
        .FirstOrDefaultAsync(f => f.Id == id, ct);
    return funko;
} //
```

Por otro lado, Supplier y Funko se encuentran almacenados en bases de datos diferentes. Por este motivo, la relación se establece mediante una referencia al identificador del proveedor (SupplierId), sin disponer de una clave foránea que conecte ambas bases de datos. Esto implica que, para recuperar la información completa de un Funko junto con sus datos de proveedor, es necesario realizar una consulta adicional a MongoDB.

La otra opción que valoramos fue devolver únicamente una referencia o ruta al endpoint donde se podría consultar la información del proveedor. Sin embargo, esto obligaría al cliente a realizar una segunda petición a la API para obtener esos datos.

Por este motivo, decidimos incluir directamente la información del proveedor en la respuesta del Funko, evitando así una petición adicional y simplificando el trabajo del cliente.

```
var result = await _supplierService.GetSupplierByIdAsync(cached.SupplierId)
    .Map(cached.ToDto)
    .Tap(funkoDto => _logger.LogInformation(
        "[SERVICE-GET] Funko con ID {Id} recuperado correctamente desde la cache.", id));

return result; // Devuelve un Result<FunkoDto, DomainError>
```

Esta decisión tiene ventajas, ya que permite obtener toda la información necesaria en una única respuesta. No obstante, también presenta un inconveniente: se envían datos adicionales que quizá no siempre sean necesarios, lo que supone cierto overhead (sobrecarga de información) en las respuestas de la API.

11.1 Validación de datos

Para la validación hemos optado por utilizar los DTOs de las solicitudes junto con anotaciones de validación (Data Annotations). Esta decisión se debe principalmente a dos motivos.

En primer lugar, comenzamos utilizando este sistema desde el inicio del desarrollo y ASP.NET Core proporciona herramientas integradas que permiten validar automáticamente los datos recibidos en cada petición. En segundo lugar, nuestros modelos tienen una complejidad relativamente baja, por lo que las reglas de validación necesarias tampoco son demasiado complejas.

No obstante, no descartamos utilizar en el futuro una solución más especializada, como FluentValidation, si aumenta la complejidad de los modelos y las reglas de validación requieren una gestión más avanzada.

```
public record CategoryCreateRequestDto
(
    [param: Required(ErrorMessage = "Nombre requerido")]
    [param: MinLength(2)]
    [param: MaxLength(100)]
    string Name,
    [param: Required(ErrorMessage = "Descripción requerido")]
    [param: MinLength(2)]
    [param: MaxLength(100)]
    string Description
);
```

11.2 Relaciones entre clases

Como se ha mencionado anteriormente, las entidades están relacionadas entre sí, y esto también se refleja en la estructura de sus modelos.

En el caso de Category y Funko, la relación se establece mediante una asociación de uno a muchos, configurada a través de Fluent API. Esto permite definir las relaciones entre ambas entidades y establecer las restricciones correspondientes en la base de datos, manteniendo la integridad de los datos.

```
//<summary> Relación con categoria </summary>
entity.HasOne(f => f.Category)
    .WithMany(c => c.Funkos)
    .HasForeignKey(f => f.CategoryId);

entity.HasQueryFilter(f => !f.IsDelete);
```

Por otro lado, la relación entre Supplier y Funko funciona de forma diferente, ya que ambas entidades se almacenan en bases de datos distintas. Por este motivo, no existe una relación directa entre ellas a nivel de base de datos. En su lugar, cada Funko almacena el identificador del proveedor (SupplierId), que posteriormente se utiliza como referencia para recuperar su información desde Mongodb.

```
18 referencias
public string SupplierId {get; set;} = string.Empty;
4 referencias
```

11.3 Paginación

Para la paginación de la información utilizamos un DTO genérico que contiene los elementos recuperados de la base de datos junto con información adicional (metadata), como el número de página actual, el total de registros y la cantidad de páginas disponibles. De esta forma, conseguimos mantener un formato de respuesta uniforme que podemos reutilizar con los distintos modelos de la aplicación.

Estas respuestas, al igual que otros datos consultados frecuentemente, pueden almacenarse en Redis para reducir las consultas a las bases de datos. Sin embargo, esto plantea un problema relacionado con la invalidación de la caché, ya que cada vez que se modifica la información es necesario actualizar o eliminar los datos almacenados para evitar devolver información desactualizada.

En el caso de la paginación, hemos optado por almacenar principalmente la primera página, ya que suele contener los registros más recientes y es la que tiene mayor probabilidad de ser consultada. En cambio, almacenar listados completos resulta menos conveniente, especialmente cuando los datos cambian con frecuencia, porque obliga a invalidar la caché continuamente y puede generar información desactualizada.

12 Comportamiento del sistema

La aplicación funciona mediante una API REST que recibe peticiones HTTP de los clientes y las procesa a través de una serie de componentes organizados por responsabilidades.

En primer lugar, las peticiones pasan por los mecanismos de autenticación y autorización, donde se comprueba la validez del token JWT y los permisos asociados al usuario. Una vez superadas estas comprobaciones, la petición llega al controlador correspondiente, que se encarga de recibir los datos y delegar su procesamiento en la capa de servicios.

Los servicios contienen la lógica necesaria para ejecutar cada operación y se comunican con los repositorios para acceder a las diferentes fuentes de almacenamiento. Durante las consultas, Redis actúa como sistema de caché para reducir los accesos a las bases de datos. Si la información no está disponible, se recupera desde la fuente correspondiente y, cuando procede, se almacena en caché.

Por último, los resultados se preparan mediante los DTOs de respuesta y se devuelven al cliente con el código HTTP correspondiente. En caso de producirse algún error durante el procesamiento, el middleware global de excepciones se encarga de gestionar la situación y proporcionar una respuesta controlada.

13 API REST y endpoints

El sistema expone cuatro controladores, cada uno encargado de gestionar las operaciones correspondientes a una de las entidades principales del negocio: Auth, Funkos, Suppliers y Categories.

Cada controlador dispone de diferentes endpoints para realizar las operaciones disponibles. Algunos de estos endpoints están protegidos mediante autorización por roles, de manera que determinadas acciones solo pueden ser ejecutadas por usuarios con los permisos adecuados.

A continuación, se describe la responsabilidad de cada controlador, sus principales operaciones y las restricciones de acceso aplicadas.

13.1 AuthController

http://localhost:5000/api/auth			
Se encarga de gestionar la autenticación de los usuarios. Expone dos endpoints públicos: Login y Signup. El primero permite iniciar sesión y obtener un token JWT para acceder a los recursos protegidos, mientras que el segundo permite registrar nuevos usuarios en la aplicación.			
Metodo	**Ruta**	**Descripción**	**Disponibilidad**
POST	/singup	Permite darse de alta como usuario por defecto solo crea con rol de tipo usuario	Público
POST	/login	Permite iniciar sesión con un usuario	Público

13.2 FunkoController

http://localhost:5000/api/funko			
Se encarga de gestionar las operaciones relacionadas con los Funkos. Expone endpoints para consultar el catálogo, obtener información de un Funko concreto, crear nuevos registros, modificar sus datos y eliminarlos. También permite gestionar aspectos como la paginación y la actualización de sus categorías y proveedores. Algunas de estas operaciones están restringidas según el rol del usuario, reservando las acciones de modificación y eliminación a los administradores.			
Metodo	**Ruta**	**Descripción**	**Disponibilidad**

GET	/	Obtiene la lista páginada de funkos, permite por medio de query elegir la página	Público
GET	/all	Obtiene la lista entera de funkos.	Público
GET	/{id:long}	Obtiene el funko especifico para ese id	Público
POST	/	Permite crear un funko	Admin
PUT	/{id:long}	Permite actualizar un funko	Admin
PATCH	/{id:long}/categoy	Permite actualizar la categoría	Admin
PATCH	/{id:long}/supplier	Permite actualizar el proveedor	Admin
DELETE	/{id:long}	Permite eliminar al funko	Admin
PATCH	/{id:long}/restore	Restaura un funko elminado	Admin

13.3 SupplierController

http://localhost:5000/api/auth			
Se encarga de gestionar los proveedores de la aplicación. Permite consultar, registrar, modificar y eliminar proveedores, almacenando su información en MongoDB.			
Metodo	**Ruta**	**Descripción**	**Disponibilidad**
POST	/	Permite crear un proveedor	Admin
GET	/	Devuelve lista paginada de proveedores	Ádmin
GET	/all	Devuelve lista complete de proveedores	Admin
GET	/{id:string}	Devuelve un proveedor especifico	Admin
PUT	/{id:string}	Actualiza un proveedor	Admin
Delete	/{id:string}	Elimina un proveedor	Admin
PATCH	/{id:string}	Restablece un proveedor eliminado	Admin

13.4 CategoryController

http://localhost:5000/api/auth			
Se encarga de gestionar los proveedores de la aplicación. Permite consultar, registrar, modificar y eliminar proveedores, almacenando su información en MongoDB.			
Metodo	**Ruta**	**Descripción**	**Disponibilidad**
POST	/	Permite crear una categoría	Admin
GET	/	Devuelve lista paginada de categorías	Ádmin
GET	/all	Devuelve lista complete de categorías	Admin
GET	/{id:string}	Devuelve una categoría especifica	Admin
PUT	/{id:string}	Actualiza una categoría	Admin
Delete	/{id:string}	Elimina una categoría	Admin
PATCH	/{id:string}	Restablece una categoria	Admin

14 Control de concurrencia.

Uno de los principales problemas en los sistemas con acceso concurrente es mantener la coherencia de los datos cuando varios usuarios intentan modificar un mismo registro al mismo tiempo. Para evitar este problema, utilizamos RowVersion, un mecanismo de control de concurrencia optimista que permite identificar la versión de un registro.

```
public record FunkoDto(
    long Id,
    string Name,
    string Franchise,
    int Number,
    CategoryDto Category,
    bool Exclusive,
    double Price,
    string? Image,
    SupplierDto Supplier,
    uint RowVersion

);
```

Cada vez que se intenta actualizar un registro, se compara la versión almacenada en la base de datos con la que envía el cliente. Si ambas coinciden, se permite realizar la modificación; en caso contrario, significa que otro usuario ha modificado previamente ese registro y se rechaza la operación para evitar sobrescribir los cambios.

De esta forma, conseguimos evitar actualizaciones accidentales y mantener la integridad de la información.

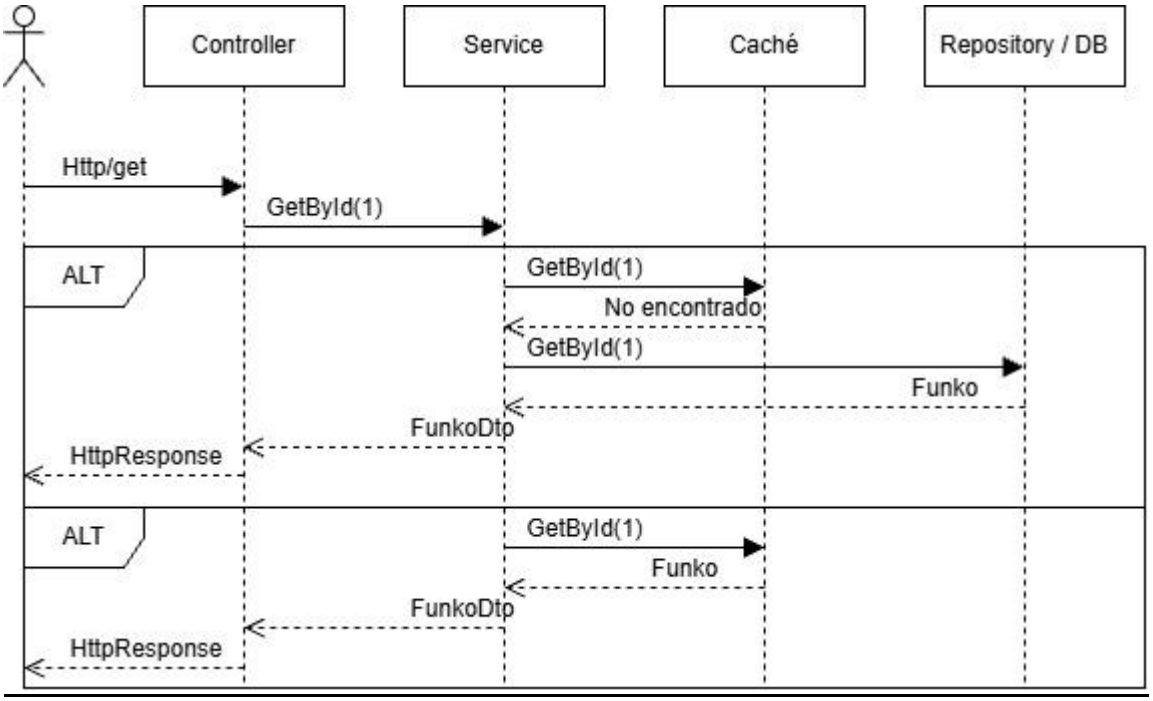
15 Servicio de almacenamiento de imágenes

Se encarga de gestionar el almacenamiento de las imágenes asociadas a los Funkos mediante el sistema de archivos del servidor. Para ello, dispone de una interfaz IStorageService y una implementación FileSystemStorageService, que permite guardar y eliminar archivos, además de comprobar su existencia. También incorpora validaciones sobre el tamaño, la extensión y el tipo de contenido de las imágenes, evitando almacenar archivos que no cumplan los requisitos establecidos. Las imágenes se almacenan dentro de wwwroot/uploads/funkos, permitiendo posteriormente acceder a ellas mediante su ruta pública.

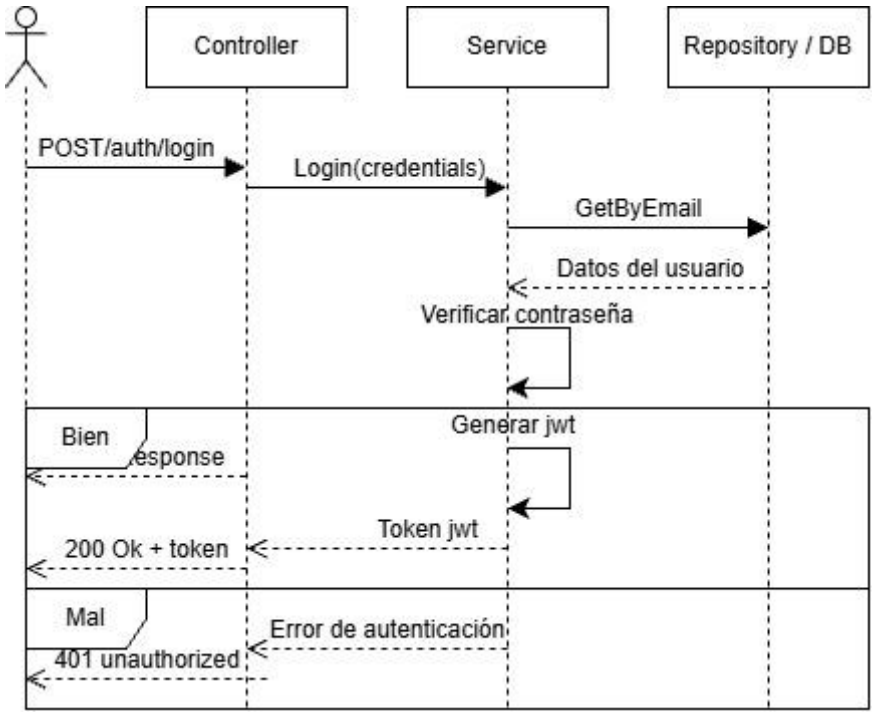
```
"Storage": {
  "UploadPath": "uploads",
  "MaxFileSize": 5242880,
  "AllowedExtensions": [".jpg", ".jpeg", ".png", ".gif"],
  "AllowedContentTypes": ["image/jpeg", "image/png", "image/gif"]
},
"Api": {
```

16 Diagramas de secuencia – CRUD

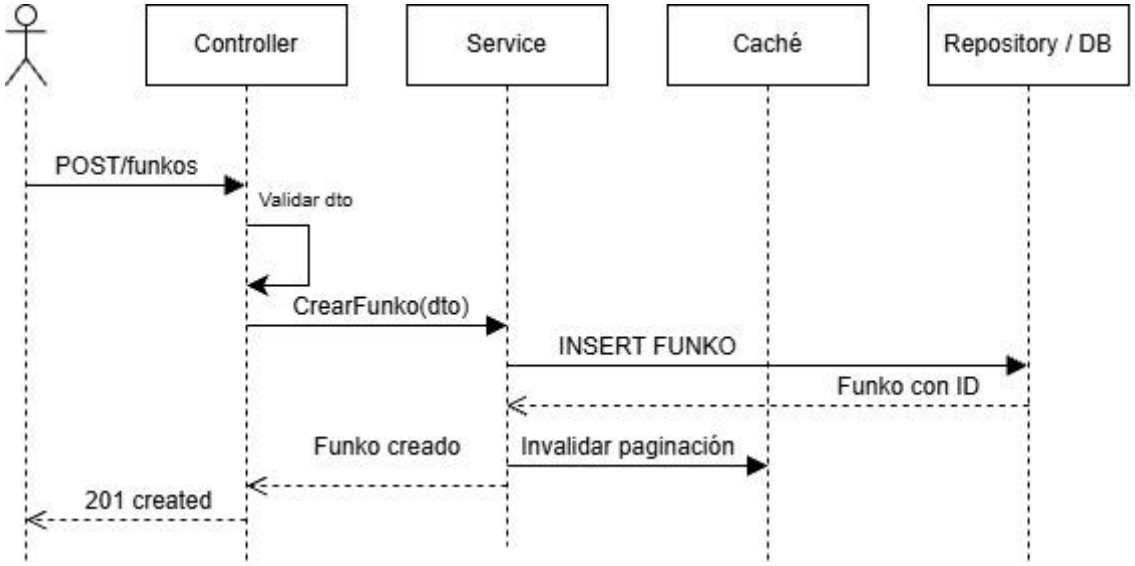
Busqueda en caché



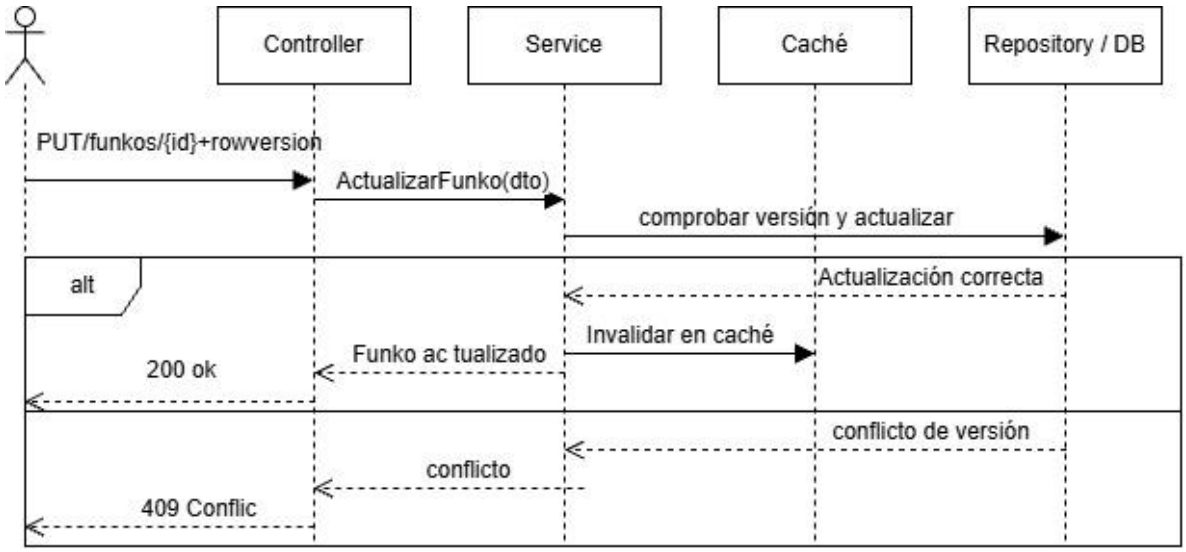
Login



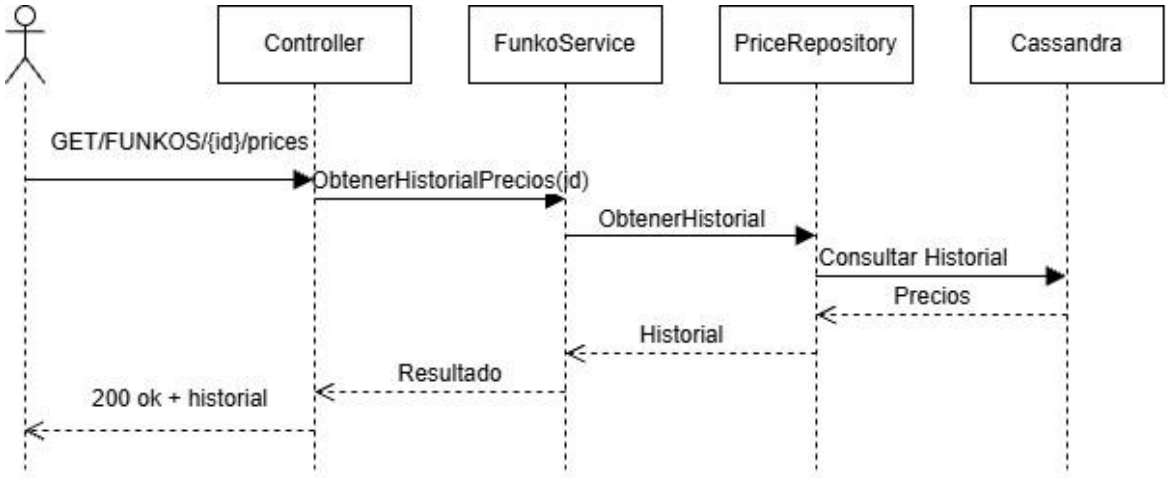
Crear funko



Actualización con row versión;



Consultar Historial:



17 Testing y pruebas

Para comprobar el correcto funcionamiento de la aplicación, se plantea una estrategia de testing basada en pruebas unitarias y de integración. Se utiliza el patrón AAA (Arrange, Act, Assert) para organizar las pruebas en tres fases: preparar los datos y dependencias, ejecutar el método que se quiere comprobar y verificar que el resultado obtenido es el esperado.

Además, se contempla el uso de Mocks para simular dependencias externas, como repositorios o servicios, permitiendo probar cada componente de forma aislada sin necesidad de acceder a las bases de datos reales.

Por otro lado, Testcontainers permite levantar instancias temporales de las bases de datos y otros servicios mediante contenedores Docker, facilitando la ejecución de pruebas de integración en un entorno similar al real.

Esta combinación permite detectar errores, comprobar el comportamiento de los distintos componentes y aumentar la fiabilidad de la aplicación.

Herramientas

NUnit	Estructura y ejecución de pruebas
Mocks	Simular repositorio, caché y storage
FluentAssertions	Comprobar resultados de forma legible.
AAA	Arrange, Act, Assert
Test Containers	Para pruebas con contenedores

Posibles comprobaciones en los test.

- Autenticación: registro, inicio de sesión y validación de JWT.
- CRUD: creación, consulta, modificación y eliminación de Funkos, categorías y proveedores.
- Validaciones: campos obligatorios, formatos e identificadores incorrectos.
- Paginación y filtros: comprobar resultados y parámetros.
- Redis: comprobar la recuperación, actualización e invalidación de la caché.
- Imágenes: validar formatos, tamaños y eliminación de archivos.
- Bases de datos: comprobar la persistencia y recuperación de información.

18 Documentación del código

Hemos utilizado XMLDoc para documentar las principales clases, interfaces y métodos de nuestro proyecto. Mediante comentarios XML describimos sus funcionalidades, parámetros y valores de retorno, facilitando así la comprensión y el mantenimiento del código.

Además, podemos consultar fácilmente las descripciones de cada elemento mientras desarrollamos, lo que nos ayuda a trabajar de forma más organizada y a comprender mejor nuesto código.

```
/// <summary>
/// Obtiene un Funko por su identificador.
/// </summary>
/// <param name="id">Identificador del Funko.</param>
/// <returns>Resultado con el funko o el error correspondiente.</returns>
```

18.1 Swagger y documentación

Se ha integrado Swagger y OpenAPI para proporcionar una documentación interactiva de los endpoints REST de la aplicación.

Esta herramienta permite consultar los controladores disponibles, conocer las rutas, los métodos HTTP, los parámetros necesarios y los modelos utilizados en las peticiones y respuestas.

También se han incorporado descripciones y documentación de las operaciones, sus parámetros y los posibles códigos de respuesta.

Además, se ha configurado el esquema de seguridad JWT Bearer, incorporando la opción Authorize en Swagger UI. Esto permite introducir un token JWT y probar directamente los endpoints protegidos, respetando los permisos establecidos mediante los roles de usuario.

```
[HttpPost("signup")]
[SwaggerOperation(
    Summary = "Registrar un usuario",
    Description = "Registra un nuevo usuario en la aplicación."
)]
[SwaggerResponse(201, "Usuario registrado correctamente.", typeof(AuthResponse))]
[SwaggerResponse(400, "Los datos proporcionados no son válidos.")]
[SwaggerResponse(409, "El usuario ya existe.")]
2 referencias
public async Task<ActionResult<AuthResponse>> Signup(
    [FromBody] SingupRequest request)
{
    var result = await authService.SignupAsync(request);

    return result.IsSuccess
        ? StatusCode(StatusCodes.Status201Created, result.Value)
        : result.Error.ToHttpResult<AuthResponse>();
}
```

De esta forma, Swagger facilita tanto la comprensión de la API como su exploración y validación durante el desarrollo.

19 Funcionalidades adicionales e integraciones

Además de las funcionalidades principales de gestión de la API REST, se han incorporado diferentes tecnologías y servicios que amplían las posibilidades de la aplicación, mejoran la comunicación con los clientes y facilitan tanto su mantenimiento como su utilización.

19.1 GraphQL

Se ha integrado GraphQL mediante HotChocolate como alternativa a las consultas tradicionales de la API REST. Las consultas se centran en consultas por lo que aprovechamos la parte de las querys para consultas más adaptables.

Esta implementación permite realizar consultas personalizadas sobre los recursos de la aplicación, evitando recuperar información que el cliente no necesita.

Se han definido los siguientes tipos:

- Funko: información principal de los Funkos.
- Category: información de las categorías.
- Supplier: información de los proveedores.
- PriceHistory: historial de precios de los Funkos.

Entre las consultas disponibles se encuentran la obtención de todos los Funkos, la búsqueda por identificador y la consulta de categorías y proveedores.

También se han incorporado mecanismos de paginación, filtrado y ordenación, además de resolvers específicos para recuperar información relacionada, como los proveedores y el historial de precios.

Es importante destacar que GraphQL se utiliza exclusivamente para operaciones de lectura (queries). Las operaciones de creación, modificación y eliminación continúan realizándose mediante los endpoints REST.

Para facilitar las pruebas se utiliza Banana Cake Pop, la interfaz de exploración y ejecución de consultas GraphQL.

19.2 Websocket/signalR

Se ha incorporado un sistema de comunicación en tiempo real mediante SignalR, utilizando WebSockets como mecanismo de transporte para mantener una comunicación bidireccional entre el servidor y los clientes conectados.

La implementación se organiza mediante dos componentes principales:

FunkoHub: hub encargado de gestionar las conexiones y la comunicación con los clientes.

NotificationService: servicio responsable de centralizar el envío de notificaciones.

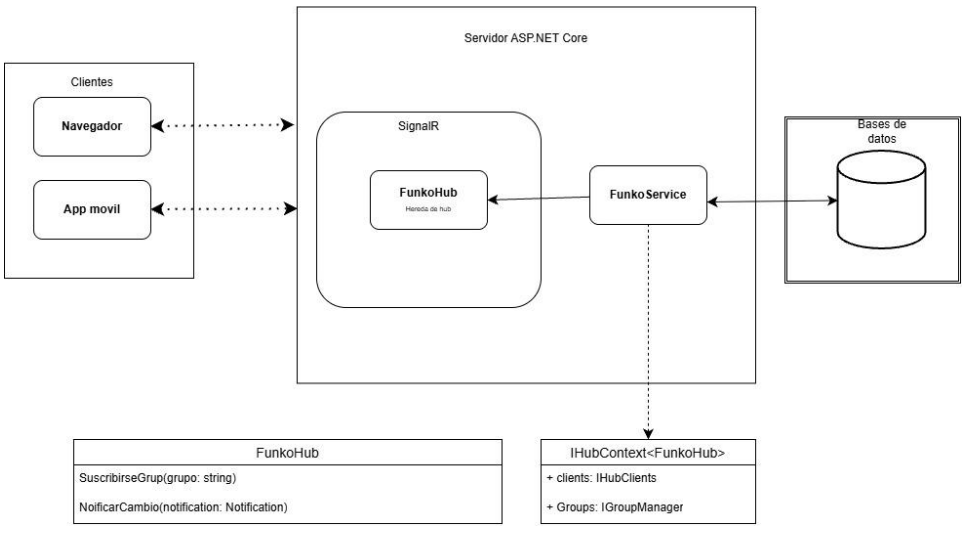
El sistema permite notificar a los administradores diferentes eventos producidos durante la ejecución de la aplicación:

- Creación de recursos.
- Actualización de recursos.
- Eliminación de recursos.
- Restauración de recursos.
- Errores no controlados.

Además, se ha integrado la autenticación JWT para controlar el acceso a las conexiones y dirigir las notificaciones al grupo de administradores correspondiente.

El endpoint de conexión del **hub** es /hubs/funkos.

Esta funcionalidad permite conocer los cambios relevantes de la aplicación sin necesidad de realizar peticiones periódicas al servidor.



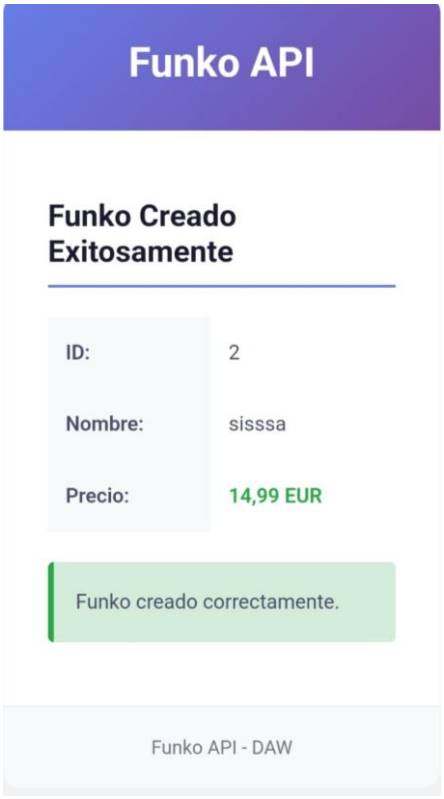
19.3 Servicio de email

Se ha desarrollado un servicio de envío de correos electrónicos utilizando MailKit, encargado de realizar la comunicación con un servidor SMTP.

La implementación se ha separado en diferentes componentes para facilitar su mantenimiento y evitar que el envío de correos bloquee la ejecución de las operaciones principales de la API.

Los componentes principales son:

- MailKitEmailService: implementa la lógica de conexión SMTP y envío de mensajes.
- SmtpSettings: contiene los parámetros necesarios para configurar la conexión SMTP.
- EmailQueueService: administra la cola de correos pendientes de envío.
- EmailBackgroundWorker: procesa los mensajes de la cola en segundo plano.



La configuración SMTP se centraliza en la sección Smtp de appsettings.json y se obtiene mediante el patrón de opciones de ASP.NET Core (IOptions<SmtpSettings>).

De esta manera, el envío de correos se realiza de forma desacoplada respecto a las operaciones principales de la API, permitiendo procesar los mensajes en segundo plano.

Análisis Económico

En este apartado se realiza un análisis económico del proyecto, teniendo en cuenta tanto los costes de las herramientas e infraestructuras utilizadas como el esfuerzo dedicado durante su desarrollo. Además, se incluye una estimación de los recursos y costes que podrían ser necesarios para incorporar futuras mejoras y ampliar las funcionalidades de la aplicación.

Estimación de costes de infraestructura y herramientas		
Visual Studio Code	Editor de código	Coste estimado
GitHub	Control de versiones y alojamiento del repositorio	0 €
Docker	Ejecución de servicios mediante contenedores	0 €
PostgreSQL	Base de datos relacional	0 €
MongoDB	Base de datos documental	0 €
Redis	Sistema de caché	0 €
Cassandra	Almacenamiento del historial de precios	0 €
Bruno	Pruebas de la API REST	0 €

Los costes se han estimado considerando el uso de versiones gratuitas y un entorno de desarrollo local. No se incluyen posibles costes de alojamiento o despliegue en producción.

Estimación de costes de infraestructura y herramientas	
Concepto	Estimación
Número de integrantes	2
Duración aproximada	8 días
Dedicación diaria por persona	4 horas
Horas dedicadas por persona	32 horas
Horas totales de desarrollo	64 horas
Coste estimado por hora	20 €/h
Coste laboral teórico	1280 €

20 Conclusión

A lo largo del desarrollo de este proyecto hemos podido poner en práctica los conocimientos adquiridos durante el curso, trabajando con diferentes tecnologías, bases de datos y patrones de diseño. La implementación de una API REST con una arquitectura por capas nos ha permitido comprender mejor cómo organizar y estructurar una aplicación real.

Además, trabajar con diferentes sistemas de almacenamiento, implementar mecanismos de seguridad y gestionar la caché nos ha supuesto un reto que nos ha ayudado a ampliar nuestros conocimientos y mejorar nuestras habilidades como desarrolladores. En definitiva, ha sido una experiencia enriquecedora que nos ha permitido consolidar lo aprendido y conocer nuevas posibilidades de desarrollo.